AI Agent 应用工具开发 ｜ 10 周系统化学习路线

（求职向 · 纯应用层 · 哈工大 CS 大三定制版）

一、路线说明

本路线专注 AI Agent **应用层/工具层开发**，不涉及大模型预训练，不涉及底层框架内核开发，不偏向科研算法。对齐全网绝大多数「大模型应用开发、AI 应用工程、Agent 开发实习」岗位通用 JD 能力要求。

与纯应用路线的区别： 在保持落地能力的基础上，补充必要的原理认知层（Transformer 推理直觉、Embedding 选型逻辑、CoT/ReAct 为什么有效），使你在面试中不仅能讲"我做了什么"，还能答"为什么这么做"——这是 CS 科班和转专业选手的核心分野。

定位： 会工程、会落地、会做可上线的 Agent 业务功能，同时能讲清楚背后的技术决策逻辑。

适配岗位关键词： 大模型应用开发、LLM 工程开发、AI Agent 开发、智能体应用研发、AI 应用工程实习。

整体周期： 10 周（相比常见 8 周版本，增加 Prompt Engineering 独立模块 + 原理认知层 + 开源项目阅读，时间更诚实可执行）。

适用起点： 会基础 Python 语法即可起步；不会脚本、不会调 API 不是障碍——第 1 周专门补足，其余能力全部在项目里现学现用。

时间假设： 每日可投入 3-5 小时（兼顾课业），周末可集中 6-8 小时。如果每天能投入更多，可压缩至 8 周。

二、整体能力目标（10 周学完可掌握）

工程能力：

- 掌握 Python + FastAPI 搭建大模型后端服务，具备可用工程基础（环境管理、脚本化、异常处理）
- 完成 RAG 检索增强链路开发，解决知识库问答场景（作为 Agent 内部工具落地）
- 掌握 Function-Calling、ReAct 范式，理解 Agent 运行逻辑
- 熟练使用 LangGraph 实现 Agent 编排（状态、节点、循环、记忆、分支）
- 具备 Agent 工程化能力：容错重试、日志观测、Docker 部署

原理认知（面试加分层）：

- 能画出 Transformer 推理流程，讲清 Attention 在做什么（不需要手推公式）
- 理解 CoT、ReAct、Function-Calling 的本质区别与适用场景
- 能说清 Embedding 模型选型逻辑（BGE vs OpenAI Embedding vs 其他，什么场景用什么）
- 理解 MCP 协议的设计动机与基本架构

Prompt 能力：

- 掌握系统化的 Prompt Engineering 方法论，能独立优化 RAG 和 Agent 的效果

最终产出：

- 1 个高含金量、**有垂直场景**、可部署、可答辩的完整 Agent 项目，覆盖 90%+ 应用岗面试考点
- 1 套技术博客/笔记（发布在 GitHub），作为面试"真做了"的证据链

三、分阶段 10 周学习路线

阶段 0：补基础 + 调 API（第 1 周）

核心目的： 把「会写基础代码」升级为「能写脚本 + 会调 API + 会用 Git」，这是后续一切的地基。

学习内容

Python 环境与工程化：

- venv / conda 环境管理，搞懂「为什么换环境跑不起来」
- pip / requirements.txt / .env 环境变量管理

脚本化：

- 接收命令行参数（argparse / sys.argv）
- 读写文件（JSON、TXT、CSV）
- 把一次性代码变成可复用脚本

Git 基础（边用边学，不开专题）：

- init / add / commit / push / pull / branch 六个命令够用

- 从第 1 天起所有代码 Git 管理，养成习惯

调第一个模型 API：

- requests 发请求、读 JSON、取回复
- 理解 API Key 管理、请求频率限制、错误码处理
- **至少调通两个不同的模型 API**（如 OpenAI / DeepSeek / 通义千问），感受差异

FastAPI 起步：

- GET / POST 路由
- 请求体校验（Pydantic）
- 统一返回体设计
- SSE 流式响应（大模型必备）

流式对话：

- 一个 token 一个 token 打印输出
- 理解 streaming 与非 streaming 的区别与适用场景

本阶段明确跳过： async 深挖、Docker（阶段 4 再学）、复杂 Git 工作流。

阶段产出

1. 一个带 SSE 流式响应的 FastAPI 后端骨架（可复用模板）
2. Git 仓库建好，后续所有代码在此基础上迭代

简历可复用描述

> 熟练使用 FastAPI 搭建后端服务，掌握 Python 环境管理与脚本化开发，具备大模型 SSE 流式响应开发能力，遵循标准化工程编码与异常处理规范。

博客/笔记任务

写一篇：《从零调通大模型 API 到 FastAPI 流式输出：踩坑记录》（500-1000 字即可，重点记录踩了什么坑、怎么解的）

阶段 1：Prompt Engineering 专项（第 2 周前半）

核心目的： Prompt 是 RAG 和 Agent 效果的最大变量，必须在动手做 RAG 之前建立系统认知，否则后续所有调优都是盲调。

学习内容

Prompt 基础范式：

- Zero-shot / Few-shot / Chain-of-Thought（CoT）
- 角色设定（System Prompt 的写法与作用）
- 输出格式控制（JSON 模式、分步输出、结构化约束）

Prompt 调优方法论：

- 指令清晰度 > 示例数量 > 模型能力，优先级排序
- "让模型先思考再回答"为什么有效（CoT 的直觉理解）
- Prompt 版本管理：改了什么、效果变了多少，养成记录习惯

实操练习（3-5 个小任务）：

- 任务 1：让模型从一段混乱文本中提取结构化信息（JSON 输出）
- 任务 2：写一个 System Prompt，让模型扮演特定角色并拒绝超范围问题
- 任务 3：对比 Zero-shot vs Few-shot vs CoT 在同一任务上的效果差异
- 任务 4：让模型对自己的回答打分并自我修正（Self-Refine 初体验）
- 任务 5：写一个 Prompt，让模型按指定格式输出工具调用指令（为 Function-Calling 做铺垫）

阶段产出

1. 一份个人 Prompt 模板库（至少 5 个经过验证的模板，含适用场景说明）
2. 一份对比实验记录（不同 Prompt 策略在同一任务上的效果对比）

简历可复用描述

> 掌握系统化 Prompt Engineering 方法论，具备 Zero-shot / Few-shot / CoT 等多种提示策略的实践经验，能独立完成 Prompt 调优与效果评估。

阶段 2：RAG 链路开发（第 2 周后半 – 第 3 周）

核心目的： 把 RAG 做成「能被 Agent 调用的一个工具」，同时理解 Embedding 选型逻辑，不做成独立项目。

学习内容

RAG 全链路：

- 文档加载（PDF、Markdown、TXT 等多格式支持）
- 文档清洗（去噪、格式统一）
- 分块策略（固定长度 vs 语义分块 vs 递归分块，理解各自适用场景）

- 向量化（Embedding 模型调用）
- 向量入库（Chroma，理解 HNSW 索引的基本原理——不需要推公式，知道为什么快）
- 语义检索（相似度计算、Top-K 召回）
- Prompt 组装（检索结果 + 用户问题 → 生成 Prompt）
- 生成应答

Embedding 模型选型（面试高频考点）：

- BGE 系列 vs OpenAI Embedding vs 其他开源模型
- 中文场景 vs 英文场景的选型差异
- 维度、速度、效果的 trade-off
- **动手实验：用至少两个不同的 Embedding 模型对比同一批文档的检索效果**

RAG 常见问题与优化：

- 幻觉成因：检索到了但答歪了 vs 没检索到瞎编了
- 召回不全：分块粒度不对 vs Embedding 语义漂移
- 基础优化方案：调分块大小、调 Top-K、优化检索 Prompt
- **进阶了解（不要求实现，但面试要能讲）：** Hybrid Search（关键词 + 语义混合检索）、Reranker 重排序的概念与价值

阶段产出

1. 一个封装好的 `rag_tool.py`，接口清晰，可直接被 LangGraph 作为工具调用
2. 一份 Embedding 选型对比记录（2-3 个模型，含检索效果对比数据）

简历可复用描述

> 独立完成 RAG 检索增强链路（文档解析、多策略分块、向量入库与语义检索），完成 Embedding 模型选型对比实验；封装为标准化可复用工具接口，有效缓解大模型幻觉问题。

博客/笔记任务

写一篇：《RAG 链路踩坑实录 + Embedding 选型实验》（记录分块策略对比、Embedding 效果对比的真实数据）

阶段 3：原理认知层（第 3 周穿插，共 2-3 天）

核心目的： 不是让你手推公式，而是建立「能在面试白板上画出来、能用大白话讲清楚」的直觉理解。这是 CS 科班面试的隐性门槛。

学习内容

Transformer 推理直觉（半天）：

- 输入一句话，Transformer 内部发生了什么（Tokenize → Embedding → Self-Attention → FFN → 输出概率分布）
- Self-Attention 在做什么：每个 token "看" 其他所有 token，决定该关注谁
- KV-Cache 为什么能加速推理（面试偶尔会问）
- **不需要：** 手推 QKV 矩阵乘法、背公式

CoT / ReAct / Function-Calling 的本质区别（半天）：

- CoT：让模型在输出中"显式思考"，本质是 Prompt 技巧
- Function-Calling：模型输出结构化的工具调用指令，由外部代码执行，本质是"模型 → 代码"的接口协议
- ReAct：思考（Reasoning）+ 行动（Acting）交替循环，是 Agent 的核心运行范式
- 画一张图：用户输入 → 模型思考 → 决定调工具 → 工具返回 → 模型再思考 → 最终回答

为什么 Agent 需要循环而不是单次调用（理解核心）：

- 单次调用 = 一问一答
- Agent 循环 = 持续思考、持续行动、直到任务完成
- 循环的终止条件设计：怎么判断"该停了"

MCP 协议基础（1-2 小时了解即可）：

- MCP 想解决什么问题（工具调用的标准化）
- 与 Function-Calling 的关系和区别
- 当前生态进展（哪些框架已支持）

阶段产出

1. 一份手绘/手写笔记：Transformer 推理流程图 + ReAct 循环流程图 + Function-Calling 时序图
2. 能用 3 分钟给别人讲清楚"Agent 是怎么运行的"

博客/笔记任务

写一篇：《Agent 到底是怎么"思考"的——ReAct / Function-Calling / CoT 的区别与联系》（配图，这篇面试前反复看）

阶段 4：LangGraph 王牌项目（第 4 – 6 周，核心重点）

核心目的： 对标企业主流技术栈，搭建简历王牌项目。**必须选定一个垂直场景**，而非做通用问答 Agent。

场景选择指南

选一个你能讲出"业务理解"的场景，面试时远比通用 Demo 有说服力：

推荐场景	工具组合	面试加分点
学术论文辅助阅读 Agent	RAG（论文库）+ 搜索（查相关工作）+ 总结工具	你是 CS 学生，天然有场景理解
代码审查/解释 Agent	RAG（项目文档）+ 代码分析工具 + 搜索（查 API 文档）	贴近开发岗日常
招聘 JD 匹配 Agent	RAG（简历库）+ JD 解析工具 + 匹配打分工具	贴近业务场景
课程/知识库问答 Agent	RAG（课件/教材）+ 搜索 + 知识图谱查询工具	学生场景，易获取数据

选定后不要换。 一个场景做深 > 多个场景做浅。

核心知识点

LangGraph 四要素：

- State：全局状态定义（TypedDict），所有节点共享的数据结构
- Node：业务节点（每个节点做一件事：思考、调工具、生成回复……）
- Edge：流转边（条件边：根据状态决定下一步走哪里）
- Graph：流程图编排（把 Node 和 Edge 组装成完整工作流）

ReAct 循环实现：

- 思考节点 → 判断是否需要工具 → 调用工具 → 观察结果 → 继续思考或输出
- 循环终止条件设计（最大轮数 + 模型主动终止）
- 循环中的异常处理（工具调用失败怎么办）

多轮对话：

- 状态持久化（Checkpointer）
- 上下文管理与对话记忆
- 记忆窗口设计（最近 N 轮 vs 摘要压缩）

Agent 自主规划：

- 任务拆解：复杂问题 → 子问题列表
- 工具选择：Agent 自主判断该调什么工具
- 分支判断：不同情况走不同处理路径

项目开发要求（纯手写，禁止低代码拖拽）

工具层（`tools/` 目录）：

- `rag_tool.py`：阶段 2 的 RAG 工具（直接复用）
- `search_tool.py`：联网搜索工具（Tavily / SerpAPI / 自定义）
- `custom_tool.py`：场景专属工具（如论文摘要提取、代码分析等）
- 每个工具统一接口规范：输入类型、输出类型、异常处理

图编排层（`graph/` 目录）：

- State 定义
- 各业务节点实现
- 条件边逻辑
- Graph 编排与编译

接口层（`api/` 目录）：

- FastAPI 路由
- 请求/响应模型定义
- SSE 流式接口
- 会话管理接口

配置层：

- `.env` 环境变量
- `config.py` 统一配置管理

第 4 周任务

- 确定垂直场景
- 搭建项目骨架（目录结构、配置管理）
- 实现 LangGraph 基础 ReAct 循环（单工具）
- 接入 RAG 工具，验证 Agent 能正确调用

第 5 周任务

- 接入第二、第三个工具
- 实现 Agent 自主工具选择与任务规划
- 实现多轮对话与状态持久化
- 实现条件分支逻辑

第 6 周任务

- 封装 FastAPI 接口（含流式响应）
- 端到端测试：从用户输入到最终输出，完整跑通
- 代码整理、注释补全、README 编写
- **产出 v1 版本，可运行、可演示**

阶段产出

1. 完整可运行、结构规范的 LangGraph 多工具智能 Agent 项目 v1（有垂直场景）
2. 项目 README（含架构图、使用说明、效果示例）

简历可复用描述

> 基于 LangGraph 实现 ReAct 范式智能 Agent，面向 [你的垂直场景] 场景，集成 RAG 检索、联网搜索等多类工具；实现 Agent 自主任务规划与工具选择、多轮对话状态持久化，封装标准化 REST 接口与 SSE 流式响应，支持外部业务系统快速接入。

博客/笔记任务

写一篇：《用 LangGraph 从零搭建 [场景名] Agent：架构设计与核心实现》（配架构图，这是面试时最重要的参考材料）

阶段 5：健壮性 + 工程化（第 7 – 8 周）

核心目的： 把 v1 升级为可部署的 v2，补齐工程化短板。Docker 在本阶段边用边学。

健壮性容错优化

LLM 输出异常处理：

- 输出格式不符合预期时的兼容解析（正则兜底、多格式尝试）
- 模型拒绝回答 / 输出空内容的处理
- JSON 解析失败的 fallback 策略

工具调用容错：

- 超时控制（单工具超时 + 总超时）
- 参数校验（工具收到非法参数怎么办）
- 执行失败自动重试（指数退避）
- 重试耗尽后的兜底策略（向用户说明而非静默失败）

上下文管理：

- 上下文超长自动裁剪（保留最近 N 轮 + 系统 Prompt）
- 对话记忆压缩（长对话摘要化）
- Token 计数与预警

全局兜底：

- 全局异常捕获中间件
- 友好的错误响应（不暴露堆栈，给出可读提示）
- 服务健康检查接口

全链路日志观测

日志设计：

- 每次请求分配唯一 trace_id
- 完整记录：用户输入 → Agent 思考过程 → 工具选择决策 → 工具入参/出参 → 执行耗时 → 最终输出
- 错误日志：异常类型、堆栈、上下文信息
- 日志分级：INFO（正常流程）/ WARNING（重试、降级）/ ERROR（失败）

可观测性：

- Agent 执行链路全追溯（给定 trace_id，能还原完整执行过程）
- 关键指标统计：平均响应时间、工具调用成功率、重试率

量化效果（最简版）

- 准备 20-50 个测试用例（覆盖你的垂直场景的典型问题）
- 写一段统计脚本：问答准确率、工具调用成功率、平均响应时间
- **不搭重型自动化评测体系**，但要有数据，面试时能说"我测过，准确率大约 XX%"

容器化部署

- 手写完整 Dockerfile（多阶段构建）
- docker-compose.yml（如果有多个服务）
- 环境变量注入

- 实现一键打包、一键部署、环境隔离
- 部署文档编写

阶段产出

1. 带容错、可观测、可部署的完整 Agent 项目 v2
2. 量化测试报告（简表即可）
3. 部署文档

简历可复用描述

> 对智能 Agent 系统进行工程化升级，实现工具调用超时重试与指数退避、LLM 输出兼容解析、上下文窗口自动裁剪与记忆压缩等健壮性策略；搭建基于 trace_id 的全链路日志观测体系，实现 Agent 思考、工具调用、任务执行全链路可追溯；通过量化测试验证系统效果（问答准确率 XX%、工具调用成功率 XX%）；使用 Docker 完成容器化封装，实现标准化部署交付。

阶段 6：开源项目阅读 + 极简手写 Agent（第 9 周）

核心目的： 跳出 LangGraph 看更大的世界，建立"我不只是会用框架"的认知深度。

开源项目阅读（选 1 个精读核心模块）

推荐项目（按优先级）：

1. **FastGPT**（推荐）—— 看它的工作流编排和知识库检索模块
2. **Dify** —— 看它的 Agent 策略实现和工具调用链路
3. **OpenGPTs / LangGraph 官方示例** —— 看 State 设计和复杂图编排

阅读方法（不是通读源码）：

- 只读核心链路：一个用户请求进来 → 经过哪些模块 → 最终输出
- 画出模块调用关系图
- 找到 3-5 个"和我的项目不同的设计决策"，思考为什么

极简手写 Agent 循环（脱离 LangGraph）

用纯 Python（不超过 100 行）实现一个最简 ReAct Agent：

```
while True:
    1. 把历史消息 + 工具描述发给模型
    2. 模型返回"思考 + 动作"
```

```
    3. 如果动作是"调用工具" → 执行工具 → 把结果加入历史
    4. 如果动作是"最终回答" → 输出并退出
    5. 超过最大轮数 → 强制退出
```

目的： 面试官问"不用框架你能不能写 Agent"，你能现场手撕。

基础多 Agent 概念（了解即可，不要求实现）

- Supervisor 模式：一个主 Agent 分发任务给子 Agent
- 角色分工：检索 Agent、总结 Agent、代码生成 Agent
- 和单 Agent 多工具的区别与适用场景
- **不需要动手做多 Agent 项目**，能在面试中讲清概念即可

阶段产出

1. 开源项目阅读笔记（含模块调用关系图 + 设计决策对比）
2. 极简手写 Agent（`mini_agent.py`，不超过 100 行，可运行）

博客/笔记任务

写一篇：《读了 [项目名] 源码后，我发现和我的项目有这些不同》（这篇面试时非常有用，证明你有技术视野）

阶段 7：面试冲刺（第 10 周）

核心目的： 把项目变成面试弹药，打磨表达，无需重做项目。

项目答辩话术准备

STAR 法则梳理（每个项目都用这个框架）：

- **Situation：** 什么场景/什么问题
- **Task：** 你要做什么
- **Action：** 你怎么做的（技术选型、架构设计、关键决策）
- **Result：** 效果如何（量化数据）

必须准备的 5 个问题：

1. "介绍一下你的项目"（3 分钟版本 + 1 分钟版本）
2. "为什么选 LangGraph 而不是其他框架"
3. "遇到的最大技术难点是什么，怎么解决的"
4. "如果让你重做，你会改什么"

5. "这个项目的局限性是什么"

高频八股准备

RAG 相关：

- RAG 为什么会产生幻觉？如何缓解？
- 分块大小怎么选？太大/太小分别有什么问题？
- Embedding 模型怎么选型？你对比过哪些？
- 语义检索 vs 关键词检索 vs 混合检索的区别？
- Reranker 的作用是什么？放在哪个环节？

Agent 相关：

- Function-Calling 与 ReAct 的区别？
- Agent 和 Chain 的区别？
- 上下文窗口溢出怎么办？
- Agent 陷入死循环怎么处理？
- 多 Agent vs 单 Agent 多工具，什么时候用哪个？

工程相关：

- 流式输出是怎么实现的？SSE 的原理？
- 工具调用超时了怎么办？
- 怎么做日志追溯？
- Docker 的基本原理？镜像和容器的区别？

原理相关（加分项）：

- Transformer 的推理过程？Attention 在做什么？
- CoT 为什么有效？
- Temperature / Top-P 参数的作用？
- MCP 协议想解决什么问题？

简历最终打磨

- 按各阶段的「简历可复用描述」逐条落实
- 量化数据全部填入（准确率、成功率、响应时间）
- 技术栈关键词确保覆盖 JD 高频词：LangGraph、RAG、Function-Calling、FastAPI、Docker、Agent
- GitHub 链接附上（确保 README 完整、代码整洁）

阶段产出

1. 面试弹药包（项目话术 + 高频问答，可打印随身携带）
2. 最终版简历
3. GitHub 项目主页（README 含架构图、效果演示、部署指南）

四、时间线总览（对应大三上学期）

```
第 1 周 | 阶段 0 | 补基础 + 调 API + Git
第 2 周 | 阶段 1 + 阶段 2 前半 | Prompt Engineering + RAG 开始
第 3 周 | 阶段 2 后半 + 阶段 3 | RAG 收尾 + 原理认知层
第 4 周 | 阶段 4 开始 | LangGraph 项目启动（场景确定 + 骨架 + 单工具）
第 5 周 | 阶段 4 继续 | 多工具 + 规划 + 多轮对话
第 6 周 | 阶段 4 收尾 | 接口封装 + 测试 + v1 产出
第 7 周 | 阶段 5 开始 | 容错 + 日志观测
第 8 周 | 阶段 5 收尾 | 量化测试 + Docker 部署 + v2 产出
第 9 周 | 阶段 6 | 开源项目阅读 + 手写 Agent
第 10 周 | 阶段 7 | 面试冲刺

预计完成时间：11 月中旬
投递窗口：11 月下旬开始投寒假实习，次年 2-3 月春招主力冲刺
```

五、投递关键词（BOSS 直聘 / 牛客 / 实习僧）

- AI Agent 开发实习生
- 大模型应用开发
- LLM 应用工程实习
- 智能体应用开发
- AI 应用研发实习生
- 大模型工程化开发

六、学成能力梯度对标

层级	能力点	覆盖情况
基础岗完全胜任	RAG 开发、基础 Agent、工具调用、接口开发、Docker 部署	✅
中高级岗匹配	Agent 状态编排、多工具闭环、链路观测、工程容错优化	✅
面试加分项	原理认知、Prompt Engineering、开源项目阅读、手写 Agent	✅
路线边界外	框架内核开发、模型训练/微调、自动化评测体系、混合检索/Reranker 实现	❌（可后续按需补充）

七、最终个人作品集（10 周收官）

1. **标准化 FastAPI 工程模板**（可复用所有项目）
2. **垂直场景 LangGraph 多工具智能 Agent 项目**（带容错、观测、部署、量化数据）
3. **RAG 检索工具**（内嵌于 Agent，含 Embedding 选型实验记录）
4. **极简手写 Agent**（`mini_agent.py`，面试手撕备用）
5. **Prompt 模板库**（经过验证的 5+ 个模板）
6. **技术博客/笔记 4-5 篇**（发布在 GitHub，面试证据链）
7. **开源项目阅读笔记**（含架构图与设计决策对比）
8. **全套简历项目话术与面试问答素材**

八、执行建议

1. **每天结束前花 10 分钟写学习日志**（今天学了什么、卡在哪里、明天计划什么）——这个习惯会让你 10 周后回顾时有清晰的成长轨迹
2. **卡住超过 2 小时就换方式**（查文档 → 问 AI → 看源码 → 搜 Issue → 暂时跳过），不要死磕
3. **第 4 周确定场景后就不要换了**，做深一个场景远比广撒网有价值
4. **博客/笔记不求完美，求真实**——记录踩坑过程比写教程更有面试价值
5. **第 6 周结束 v1 可运行后，就可以开始小范围投递试水**，用面试反馈倒逼后续优化方向